Practice 14. Use Hash Tables

[CSE2010] Data Structures

Department of Data Science

Overview

- Solutions for last practice
- Built-in data structures in C++ and Python
- Today’s practice

Practice 12 C++ Solution

- For binary search
  - readInput()
  - binarySearch()
- For perfect square
  - isPerfectSquare()

```cpp
int main(int argc, char* argv[]) {
  if (argc != 3) {
    cerr<<"Correct usage: [program] [input] [output]"<<endl;
    exit(1);
  }
  ifstream inFile(argv[1]);
  ofstream outFile(argv[2]);
  string line;
  while (getline(inFile, line))
  {
    char op = line[0];
    istringstream iss(line.substr(2));
    vector<int> arr;
    int size, x, index;
    switch(op)
    {
      case BINARY_SEARCH:
        tie(size, x) = readInput(inFile, iss, arr);
        outFile<<op<<" "<<size<<" "<<x<<endl;
        index = binarySearch(x, arr);
        if (index < 0)
          outFile<<"N"<<endl;
        else
          outFile<<index<<endl;
        break;
      case PERFECT_SQUARE:
        if (!(iss >> x)) {
          cerr<<"Error: invalid input"<<endl;
          exit(1);
        }
        if (isPerfectSquare(x))
          outFile<<"T"<<endl;
        else
          outFile<<"F"<<endl;
        break;
      default:
        cerr<<"Undefined operator"<<endl;
        exit(1);
    }
  }
  outFile.close();
  inFile.close();
}
```

Practice 12 C++ Solution

```cpp
pair<int, int> readInput(ifstream& inFile, istringstream& iss, vector<int>& input) {
  string line;
  int numInput, key, elem;
  input.clear();
  if (!(iss >> numInput >> key)) {
    cerr<<"Error: invalid input"<<endl;
    exit(1);
  }
  if (!getline(inFile, line)) {
    cerr<<"Error: unable to read a line"<<endl;
    exit(1);
  }
  istringstream eiss(line);
  for (int i = 0; i < numInput; ++i) {
    if (!(eiss >> elem)) {
      cerr<<"Error: invalid input value"<<endl;
      exit(1);
    }
    else {
      input.push_back(elem);
    }
  }
  return make_pair(numInput, key);
}
```

```cpp
int binarySearch(int x, vector<int>& arr) {
  int mid, l = 0, r = arr.size() - 1;
  while (l <= r) {
    mid = (l + r) / 2;
    if (x == arr[mid])
      return mid;
    else if (x < arr[mid])
      r = mid - 1;
    else
      l = mid + 1;
  }
  return -1;
}

bool isPerfectSquare(int x) {
  int square, mid, l = 0, r = x;
  while (l <= r) {
    mid = (l + r) / 2;
    square = mid * mid;
    if (square == x)
      return true;
    else if (square < x)
      l = mid + 1;
    else
      r = mid - 1;
  }
  return false;
}
```

Standard Template Library in C++

- Standard Template Library (STL) provides the built-in implementations of data structures and abstract data types.
  - Dynamic arrays
  - Stacks & Queues
  - Doubly linked lists
  - Red-black trees (a kind of self-balancing BST)
  - Hash tables
  - Heaps

Standard Template Library in C++

Name	Data Structure	Use for (or Benefits)	Don't use for (or Limitations)
std::vector	dynamic array	• Simple storage • Pushing to or popping from the end • Quick lookup by index	• Insertion/deletion in the middle
std::deque (**d**ouble-**e**nded **queue**)	vector	• Similar to std::vector • Push and pop at both ends	• Insertion/deletion in the middle
std::stack	vector or deque	• First-in last-out operations • Reversal of elements	
std::queue	vector or deque	• First-in first-out operations	
std::list	doubly linked list	• Inserting to the middle/beginning • Sorting	• Direct access
std::set	BST	• Only keys • Removing duplicates	• Simple storage • Direct access by index
std::map	red-black tree	• Key-value pairs • Lookup with keys • Removing duplicates	• Slower than unordered_map
std::unordered_map	hash table		• Sorting
std::priority_queue	heap	• Getting keys with the highest priority	

Built-In Data Structures in Python

- Python provides the implementations of data structures.
  - Dynamic arrays: lists
  - Hash tables: sets, dictionaries
  - Doubly linked lists: deques
    - from collections import deque
    - Use a deque for stack or queue
  - Heaps: heapq
    - import heapq

Built-In Data Structures in Python

Name	Data Structure	Use for (or Benefits)	Don't use for (or Limitations)
list	dynamic array	• Simple storage • Appending to or popping from the end • Quick lookup by index	• Insertion/deletion in the middle
deque (**d**ouble-**e**nded **queue**)	doubly linked list	• Push and pop operations at both ends	• Insertion/deletion in the middle
set	hash table	• Only keys • Removing duplicates	• Simple storage • Direct access by index
dictionary	hash table	• Key-value pairs • Lookup with keys • Removing duplicates	• Sorting
heapq	heap	• Getting keys with the highest priority	

Helpful Reference

- C++ cheat sheet
  - https://github.com/gibsjose/cpp-cheat-sheet/blob/master/Data%20Structures%20and%20Algorithms.md#12-vector-stdvector
- What data structure is inside std::map in C
  - https://stackoverflow.com/questions/18414579/what-data-structure-is-inside-stdmap-in-c
- Python data structure cheat sheet
  - https://intellipaat.com/blog/tutorial/python-tutorial/data-structures-with-python-cheat-sheet/

TODO for Today’s Practice

- Write a program to efficiently solve Problems 1 and 2.

1. Two Sum

- **Problem**.
  Given a list of numbers and a number k, return whether any two numbers from the list add up to k.
- Example 1
  - Input: **k = 19, nums = [5, 3, 10, 12, 9]**
  - Output: **True**
- Example 2
  - Input: **k = 11, nums = [5, 3, 10, 12, 9]**
  - Output: **False**

2. Symmetric Pairs

- **Problem**.
  Two pairs (a, b) and (c, d) are symmetric if c is equal to b and a is equal to d.
- For example, (1, 2) and (2, 1) are symmetric.
- Given an array of pairs, find all symmetric pairs in it. Assume the first elements of all pairs are distinct.
- Example
  - Input: arr =**[(1, 2), (3, 4), (5, 10), (4, 3), (10, 5)]**
  - Output: **[(3, 4), (5, 10)]**
- Hint
  - You can assume every pair is distinct (workaround)

Input File

- **T<space>[k]<space>[number_of_elements]**
  [int]<space>[int]<space>…<space>[int]
  - Write to output file “T” if there exist any two numbers that add up to k; “F” otherwise.
- **S<space>[number_of_pairs]**

$$\left.\begin{array}{c}\textbf{[int]<space>[int]}\\ \vdots \\ \textbf{[int]<space>[int]}\end{array}\right\}\textbf{[number_of_pairs]}$$

  - Write to output file every symmetric pair

Input and Output

- Each line in output file corresponds to the answer of a problem.
- Input File

```
T 19 5
5 3 10 12 9
T 11 5
5 3 10 12 9
```

Output File

```
T
F
```

```
S 4
1 3
2 1
4 1
3 1
```

```
1 3
```

Running Example

```
[hjkim@localhost hash_tables]$ g++ -std=c++11 use_hash_table.cpp -o exe
[hjkim@localhost hash_tables]$ cat input2.txt
T 19 5
5 3 10 12 9
T 11 5
5 3 10 12 9
[hjkim@localhost hash_tables]$ ./exe input2.txt output.txt
[hjkim@localhost hash_tables]$ cat output.txt
T
F
[hjkim@localhost hash_tables]$ cat input3.txt
S 4
1 3
2 1
4 1
3 1
[hjkim@localhost hash_tables]$ ./exe input3.txt output.txt
[hjkim@localhost hash_tables]$ cat output.txt
3 1
[hjkim@localhost hash_tables]$
```

Submission Guideline

- Submission: **source code, an input file, an output file**
  - Where: Assignments in LMS
  - Deadline: **23:59, June. 9th (Sunday)**

Hints

- Problem 2 (Symmetric Pairs): you can assume that every pair in input is distinct.
- General: use the built-in implementation of hash tables
  - C++: unordered_map
  - Python: dictionary